

Design

EXMEM被设计为一个动态链接库，可以通过[LD_PRELOAD](#)透明地加载到应用程序地址空间中

用户代码与EXMEM交互：

- 通过库API显式交互
- 通过系统调用隐式交互：当用户代码调用 `mmap` 时，EXMEM用 `Intel libsyscall_intercept` 拦截，接管并执行它自己定义的功能（系统调用、EXMEM 库中的其他功能）

Core Layer

负责与内核交互

EXMEM管理其绑定的应用程序在本地和远程（如NVMe SSD）上的后端存储资源，当多个EXMEM实例共享一块内存时，由global manger为每个EXMEM实例（即一个被管理的应用程序）分配一块内存，由EXMEM对这块内存进行细粒度的管理

- EXMEM拦截用户代码的 `mmap` 等系统调用（在进程的**虚拟地址空间**中预留了一段连续的地址范围，**但没有为它分配任何实际的物理内存**）
- 将应用程序用到的内存区域注册/注销到userfaultfd
 - 与mmap的其他区别：
 - 强制私有映射——强制所有内存都成为私有映射，不处理共享内存的复杂同步问题；强制匿名映射——确保这块内存是匿名的、由程序自己使用的内存；强制不预留交换空间——EXMEM将完全接管自己的交换逻辑
 - 移除内核预填充，调用自定义预填充
 - 调用 `policy_ack_vma` 函数，告知policy layer此内存的存在
- 当出现page fault时（当虚拟地址无法转换为物理地址，就会立刻触发一个**缺页异常 (Page Fault)**），userfaultfd将错误转发给EXMEM处理

管理的内存状态：

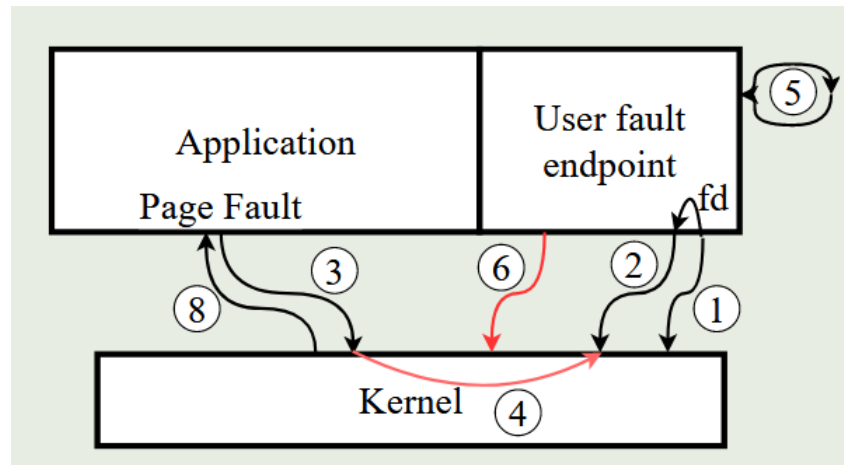
- 一个虚拟地址是否被 EXMEM 系统所追踪

```
struct user_page* find_page(uint64_t va)
{
    struct user_page *page;
    pthread_mutex_lock(&pages_lock);
    HASH_FIND(hh, pages, &va, sizeof(uint64_t), page);
    pthread_mutex_unlock(&pages_lock);
    return page;
}
```

- 页面的数据当前存放在DRAM还是磁盘
- 页面是否正在被操作
-

userfaultfd

使用进程间通信（IPC）将错误转发给用户态程序——Linux



- 应用程序向内核注册一个内存区域并接收一个**文件描述符fd**
- 处理线程Handler thread（处理故障的线程）调用 select/poll监听fd
- faulting thread触发page fault，由用户态切换至内核态
- 内核调用了 `handle_userfault` 交给 userfaultfd 相关的代码进行处理。一个待处理的消息 `uffd_msg` 结构通过该 fd 发送到handler thread，faulting thread被挂起进入阻塞状态
- handler thread接收待处理消息，通过 `ioctl` 处理page fault：
 - `UFFDIO_COPY`：将用户自定义数据拷贝到 faulting page 上
 - `UFFDIO_ZEROPAGE`：将 faulting page 置 0
 - `UFFDIO_WAKE`：用于配合上面两项中 `UFFDIO_COPY_MODE_DONTWAKE` 和 `UFFDIO_ZEROPAGE_MODE_DONTWAKE` 模式实现批量填充
- 在处理结束后handler thread发送信号唤醒 faulting thread继续工作

弊端：

- faulting thread和handler thread构成近似IPC机制

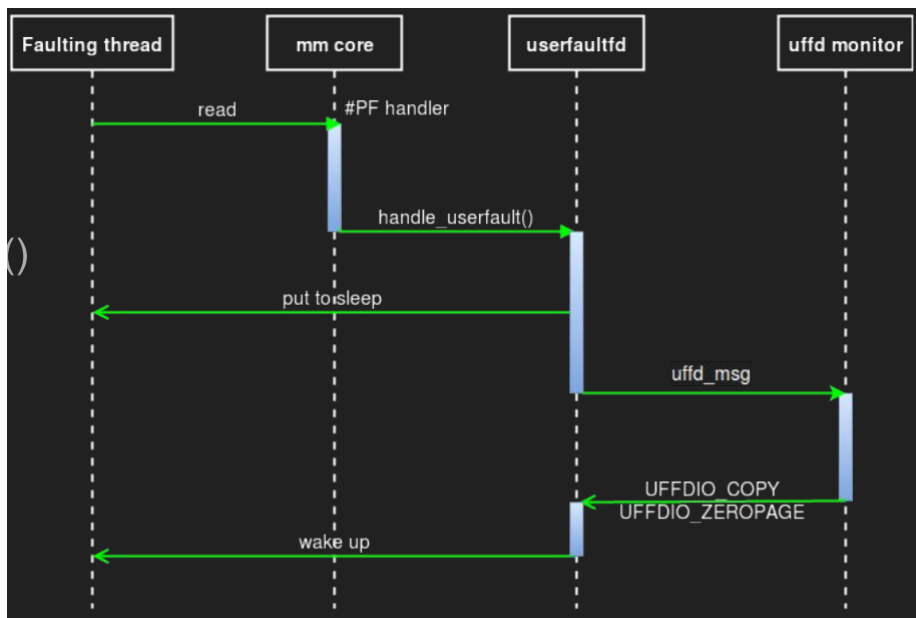
传统IPC：

进程A → `write(pipe_fd, data)` → 内核缓冲区 → `read(pipe_fd, data)` → 进程B

userfaultfd：

故障线程 → `page_fault` → 内核队列 → `read(uffd, msg)` → handler线程

- 一个 handler 线程处理所有缺页，在高并发下会成为瓶颈；若采用多个handler，userfaultfd只有一个消息队列，所有handler对其加锁读，无法发挥handler并发优势



upcall

当一个线程发生页错误时，内核会捕获这个错误，在出错线程自己的用户态栈上设置好上下文，然后将控制权直接交还给该线程在用户空间预先注册好的页错误处理函数

exokernel的upcall机制：

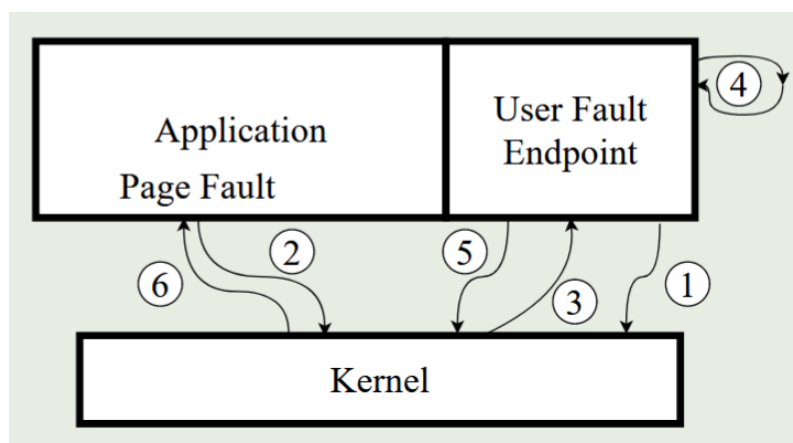
- 当发生页错误时，保留寄存器
- 加载异常信息
- 直接跳转到应用程序在“异常上下文（exception context）”中预先指定的程序计数器（PC）地址

Linux仿造exokernel的upcall机制：

使用信号处理路径signing handling将错误转发给用户态程序——使用方法：Linux信号处理机制

```

// 理想的纯upcall（如Exokernel）
void kernel_page_fault_handler(unsigned long fault_addr) {
    // 直接修改用户态执行上下文
    current->regs.ip = user_fault_handler;
    current->regs.di = fault_addr; // 传递参数
    // 返回用户态时直接执行handler
}
  
```



- 应用程序向内核注册一个内存区域并注册upcall handler

```
void uswap_register_handler(int sig)
```

```

{
    struct sigaction action;
    LOG("Registering a signal handler\n");

    action.sa_flags = SA_RESTART;
    action.sa_handler = NULL;
    // 当指定的信号发生时，请调用 uswap_sig_handler
    action.sa_sigaction = uswap_sig_handler;
    action.sa_flags |= SA_SIGINFO;
    sigemptyset(&action.sa_mask);
    // 将配置好的 action 结构体与指定的信号 sig 关联起来
    int ret = sigaction(sig, &action, NULL);
    if (ret != 0)
    {
        perror("sigaction failed \n");
    }
    LOG("Signal handler for signal %d registered\n", sig);
}
// core layer
uswap_register_handler(SIGBUS);

```

- faulting thread触发page fault，由用户态切换至内核态
- 内核调用了 `handle_userfault` 交给 `userfaultfd` 相关的代码进行处理，向引发该page fault的faulting thread的私有待处理队列发送一个 SIGBUS 信号
- 当faulting thread即将从内核态返回用户态时，读取待处理队列中的SIGBUS信号
- 调用 `arch_do_signal_or_restart` --> `handle_signal` 函数，构建信号帧 (`setup_rt_frame`)
 - 在故障线程的**用户态栈**上创建一个包含完整寄存器上下文和 `siginfo_t` 信息的**信号帧**
 - 修改保存在内核中的返回上下文
 - 将返回地址 (`regs->ip`) 修改为 `upcall_handler` 的地址，并将栈指针 (`regs->sp`) 指向刚刚创建的信号帧

```

static int __setup_rt_frame(int sig, struct ksignal *ksig,
                           sigset_t *set, struct pt_regs *regs)
{
    struct rt_sigframe __user *frame;
    void __user *restorer;
    void __user *fp = NULL;

    // 在当前线程的用户态栈上计算出一个安全的位置，用来存放即将创建的“信号帧”
    frame = get_sigframe(&ksig->ka, regs, sizeof(*frame), &fp);

    if (!user_access_begin(frame, sizeof(*frame)))
        return -EFAULT;

    unsafe_put_user(sig, &frame->sig, Efault);
    unsafe_put_user(&frame->info, &frame->pinfo, Efault);
    unsafe_put_user(&frame->uc, &frame->puc, Efault);

    /* Create the ucontext. */
    if (static_cpu_has(X86_FEATURE_XSAVE))
        unsafe_put_user(UC_FP_XSTATE, &frame->uc.uc_fflags, Efault);
}

```

```

else
    unsafe_put_user(0, &frame->uc.uc_flags, Efault);
    unsafe_put_user(0, &frame->uc.uc_link, Efault);
    unsafe_save_altstack(&frame->uc.uc_stack, regs->sp, Efault);

    /* Set up to return from userspace. */
    // 由内核提供的、标准的恢复函数 __kernel_rt_sigreturn
    // __kernel_rt_sigreturn从用户态栈上拷贝回之前保存的 ucontext_t 信息，并用
    它来恢复所有的CPU寄存器
    restorer = current->mm->context.vdso +
        vdso_image_32.sym__kernel_rt_sigreturn;
    if (ksig->ka.sa.sa_flags & SA_RESTORER)
        restorer = ksig->ka.sa.sa_restorer;
    // 让pretcode指向__kernel_rt_sigreturn，执行完sig handler后跳转到这里
    unsafe_put_user(restorer, &frame->pretcode, Efault);

    /*
        * This is movl $__NR_rt_sigreturn, %ax ; int $0x80
        *
        * WE DO NOT USE IT ANY MORE! It's only left here for historical
        * reasons and because gdb uses it as a signature to notice
        * signal handler stack frames.
    */
    // 将完整的上下文信息（包括所有寄存器的值、siginfo_t 等）拷贝到刚刚计算出的用户
    态栈地址上
    // --在执行完sig handler后，要根据此context恢复到原先faulting thread陷入内
    核处
    unsafe_put_user(((u64 *)&rt_retcode), (u64 *)frame->retcode,
        Efault);
    unsafe_put_sigcontext(&frame->uc.uc_mcontext, fp, regs, set,
        Efault);
    unsafe_put_sigmask(set, frame, Efault);
    user_access_end();

    if (copy_siginfo_to_user(&frame->info, &ksig->info))
        return -EFAULT;

    /* Set up registers for signal handler */
    // 将栈顶指针，指向刚刚在用户态栈上创建的那个信号帧！！！！！！！！
    regs->sp = (unsigned long)frame;
    // 将下一条要执行的指令地址，强行设置为信号处理函数的地址！！！！！！
    regs->ip = (unsigned long)ksig->ka.sa.sa_handler;
    regs->ax = (unsigned long)sig;
    regs->dx = (unsigned long)&frame->info;
    regs->cx = (unsigned long)&frame->uc;

    regs->ds = __USER_DS;
    regs->es = __USER_DS;
    regs->ss = __USER_DS;
    regs->cs = __USER_CS;

    return 0;
Efault:
    user_access_end();
    return -EFAULT;

```

```
}
```

- faulting thread触发page fault，由用户态切换至内核态
- 内核调用了 `handle_userfault` 交给 `userfaultfd` 相关的代码进行处理，向引发该page fault的 faulting thread发送一个 SIGBUS 信号
- 内核在faulting thread设置新的执行上下文
- faulting thread返回用户空间，跳到一开始注册的upcall handler执行，upcall handler调用policy layer的方法，采取不同的内存策略

```
//  
void uswap_sig_handler(int code, siginfo_t *siginfo, void *context)  
{  
    // ...  
    // 从 siginfo 结构体中获取缺页的内存地址  
    fault_addr = (uint64_t)siginfo->si_addr;  
    // ...  
  
    page = find_page(page_boundry);  
  
    // 根据页面的状态，分发给不同的处理函数  
    if(page == NULL){ // 第一次访问  
        handle_first_fault(page_boundry);  
    }  
    else if(page->swapped_out == true){ // 页面被换出  
        handle_missing_fault(page);  
    }  
    else{ // 写保护缺页  
        handle_wp_fault(page);  
    }  
    // ...  
}
```

- faulting thread从handler得到新的页面
- 继续执行

Observability Layer

跟踪内存访问，以区分经常访问（热）和不常访问（冷）数据

- page fault的故障地址
- MMU访问位和脏位

```
uint64_t pt_get_bits(struct user_page *page)  
{  
    uint64_t ret;  
    struct uffdio_page_flags page_flags;  
  
    page_flags.va = page->va;  
    assert(page_flags.va % PAGE_SIZE == 0);  
    page_flags.flag1 = PT_ACCESSED_FLAG;  
    page_flags.flag2 = PT_DIRTY_FLAG;
```

```

if (ret = ioctl(uffd, UFFDIO_GET_FLAG, &page_flags) < 0) {
    fprintf(stderr, "userfaultfd_get_flag returned < 0\n");
    assert(0);
}
ret = page_flags.res1 | page_flags.res2;

return ret;
}

```

policy layer 通过检查页面的**访问位 (Accessed Bit)**，判断 `inactive_list` 中的页面是否应该被回收

```

bits = pt_get_bits(page);
...
if ((bits & PT_ACCESSED_FLAG) == PT_ACCESSED_FLAG) {
    // 页面被访问过，将其重新激活
    ...
    enqueue_fifo(&active_list, page);
}
else {
    // 页面未被访问，是“冷”页面，将其放入待换出列表
    enqueue_fifo(&eviction_list, page);
    ...
}

```

policy layer 通过检查页面的**脏位 (Dirty Bit)**，决定是否要将页面的内容写回磁盘

```

flags = pt_get_bits(victim[nr_prepared]);
...
dirty[nr_prepared] = flags & PT_DIRTY_FLAG;
...
writeback[nr_prepared] = dirty[nr_prepared] | (!(victim[nr_prepared] -
>has_reserve));

```

- 硬件计数器

Policy Layer

Core Layer的handler调用，能够自定义策略：识别需要驱逐或降级的冷页，并选择潜在的页面进行预取或promotion

[ExtMem/src at master · SepehrDV2/ExtMem](#)

upcall优化浅析

- faulting thread和handler thread构成近似IPC机制

传统IPC:

```

进程A → write(pipe_fd, data) → 内核缓冲区 → read(pipe_fd, data) → 进程B

```

userfaultfd:

```

故障线程 → page_fault → 内核队列 → read(uffd, msg) → handler线程

```

faulting thread发生page fault后陷入内核，将错误交给 userfaultfd 处理，`uffd_msg` 被送入uffd等待队列，faulting thread被挂起进入阻塞状态

等待handler thread将错误处理完毕后，才会将faulting thread唤醒

upcall优化：page fault交由引发该错误的faulting thread处理，没有handler thread（内核修改该线程自己的执行上下文，使其返回用户态时执行SIGBUS handler），不用进行IPC，减少开销

- userfaultfd只有一个消息队列，无论有多少handler thread，都要对其加锁读，本质上是串行化的过程

upcall优化：取消消息队列，faulting thread陷入内核后 uffd 发出SIGBUS信号（说是内核发出SIGBUS信号，其实只是此时控制权在内核，而本质还是陷入内核的faulting thread）

- 因为每个线程独立，多个线程可以同时发出SIGBUS（不用竞争同一个userfaultfd的等待队列）消除了 uffd 的等待队列串行化的局限
- 同时，每个线程使用**独立的内核信号处理结构**处理SIGBUS信号，而不像传统的SIGBUS信号处理——同一个进程下的所有线程竞争同一个sig handler

Question

- upcall原本的实现
- Policy layer有哪些API（哪些API对应哪些功能）
 - `lrudisk_init(void)`: 初始化API。这是策略的入口点，在程序启动时由EXTMEM核心调用一次。它负责初始化所有页面列表（如空闲DRAM列表、空闲磁盘列表）并创建和启动所有后台工作线程（kswapd、eviction worker、prefetch worker）。
 - `lrudisk_pagefault(void)`: 页错误处理API。当应用程序发生页错误需要分配一个新页面时，EXTMEM核心会调用此函数。它负责从空闲DRAM列表中提供一个可用的物理页面。
 - `lrudisk_swapin_external(struct user_page \*page)`: 页面换入API。当应用程序访问一个已经被换出到磁盘的页面时，此函数被调用。它负责处理将该页面从磁盘读回内存的逻辑。
 - `lrudisk_track_page(struct user_page \*page)`: 页面跟踪API。当一个新页面被成功映射到内存后，核心框架调用此函数，将该页面交给策略进行管理（通常是放入`inactive_list`开始LRU跟踪）。
 - `lrudisk_remove_page(volatile struct user_page \*page)`: 页面移除API。当应用程序释放一块内存时（例如调用`munmap`），此函数被调用来将对应的页面从策略的所有列表中移除，并回收其资源。
 - `lrudisk_ack_vma(void \* vma_boundry, ...)`: VMA通知API。当EXTMEM框架拦截到`mmap`系统调用时，会调用此函数来通知策略有一个新的虚拟内存区域（VMA）需要被管理。
- 每个线程使用**独立的内核信号处理结构**——看一下claud

架构

```
// 每个进程（多线程共享）
struct signal_struct {
    // 进程级信号相关数据
    ...
};

// 信号处理动作表（多线程共享）
struct sighand_struct {
    atomic_t count; // 引用计数
```

```

spinlock_t siglock; // 保护锁（竞争点！）
wait_queue_head_t signalfd_wqh;
struct k_sigaction action[_NSIG]; // 64个信号的处理函数表
};

// 每个线程
struct task_struct {
    struct signal_struct *signal; // 指向共享的进程级信号结构
    struct sighand_struct *sighand; // 指向共享的信号处理表（关键！）
    // ExtMEM新增的（关键创新！）
    struct sighand_struct *uffd_sighand; // 每个线程独立的结构
    ...
};

```

实现逻辑

```

// 简化的内核代码逻辑

// 发送信号时
void send_signal_to_task(int sig, struct task_struct *task) {
    struct sighand_struct *sighand;

    // ExtMEM添加的判断
    if (sig == SIGBUS && is_from_userfaultfd_pagefault(task)) {
        // 使用per-thread的独立结构（无竞争！）
        sighand = task->uffd_sighand;
    } else {
        // 使用传统的共享结构（原有行为）
        sighand = task->sighand;
    }

    spin_lock(&sighand->siglock); // 现在每个线程锁自己的
    // ... 设置信号处理
    spin_unlock(&sighand->siglock);
}

```

多个线程并行

时间轴：

```

t1: Thread1 page_fault → 操作uffd_sighand1 → 设置handler → 执行handler
t1: Thread2 page_fault → 操作uffd_sighand2 → 设置handler → 执行handler
t1: Thread3 page_fault → 操作uffd_sighand3 → 设置handler → 执行handler

```

- 为什么原本的linux每个线程不使用**独立的内核信号处理结构**
 - POSIX标准要求同一个进程内，信号处理动作必须共享
 - 每个进程数百个线程的情况开销大
 - 信号触发频率低，锁竞争不明显
- 在故障线程的**用户态栈**上创建的、包含完整寄存器上下文和 `siginfo_t` 信息的**信号帧**——具体哪些信息
 - 信号本身的信息 `siginfo_t`：故障地址等
 - 线程在中断并陷入内核前的硬件状态 `ucontext_t`：通用寄存器值、栈信息、信号掩码
 - 信号处理流程控制信息：sig handler地址、信号返回码 `frame->precode`

- 关于最后一个实验的定制策略和Compressed Sparse Row (CSR)